

ROBOT EMBEDDED PROGRAMING

Lab 4 Creating and Managing ROS 2 Workspaces and Packages



Name:

Lance Quan

Michaël McCorkell

Percy Sanford

Tyler Brannon

HUMBER POLYTECHNIC

FACULTY OF APPLIED SCIENCES AND TECHNOLOGY

Objective

In this lab, you will learn how to:

- Source a ROS 2 workspace.
- Run a custom publisher node.
- Open the turtlesim simulator.
- Observe the effect of your publisher on the turtle's movement.
- Edit package files using Visual Studio Code (VS Code).

Prerequisites

- Ubuntu with ROS 2 Jazzy installed.
- Visual Studio Code installed.
- A workspace named turtle_py_ws.
- Package turtle_py_pkg created.

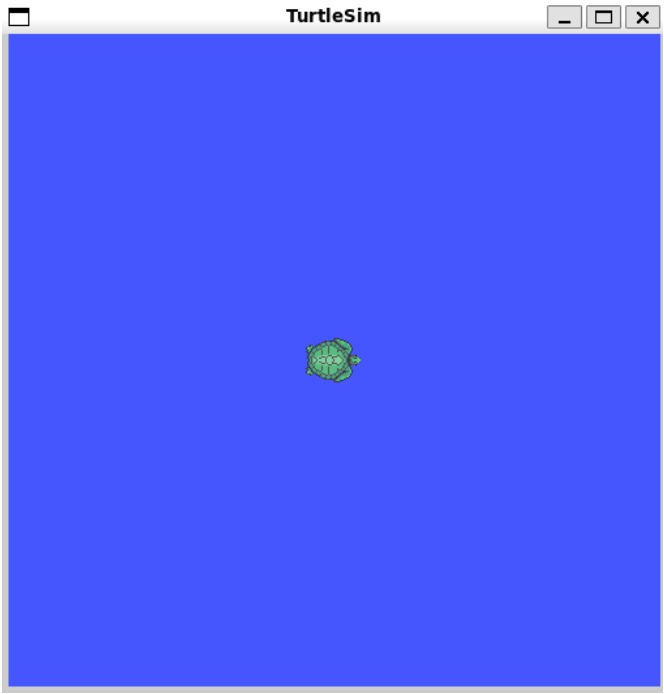
Steps 1–3: Setup

1. Verify system and ROS:

```
mystic@Zallmos:~$ ls -la /opt/ros/jazzy
total 144
drwxr-xr-x   9 root root  4096 Sep 14 14:50 .
drwxr-xr-x   3 root root  4096 Sep 14 14:49 ..
drwxr-xr-x   2 root root  4096 Sep 21 13:45 bin
drwxr-xr-x 157 root root  4096 Sep 21 13:45 include
drwxr-xr-x 110 root root 49152 Sep 21 13:45 lib
-rw-r--r--   1 root root   373 Jan 21  2026 local_setup.bash
-rw-r--r--   1 root root  3901 Feb 24  2026 local_setup.sh
-rw-r--r--   1 root root 15739 Jan 21  2026 _local_setup_util.py
-rw-r--r--   1 root root   379 Jan 21  2026 local_setup.zsh
drwxr-xr-x   8 root root  4096 Sep 21 13:45 opt
-rw-r--r--   1 root root   349 Jan 21  2026 setup.bash
-rw-r--r--   1 root root  4274 Feb 24  2026 setup.sh
-rw-r--r--   1 root root   622 Jan 21  2026 setup.zsh
drwxr-xr-x 364 root root 20480 Sep 21 13:45 share
drwxr-xr-x   4 root root  4096 Sep 14 14:49 src
drwxr-xr-x   3 root root  4096 Sep 14 14:49 tools
```

```
mystic@Zallmos:~$ printenv ROS_DISTRO
jazzy
```

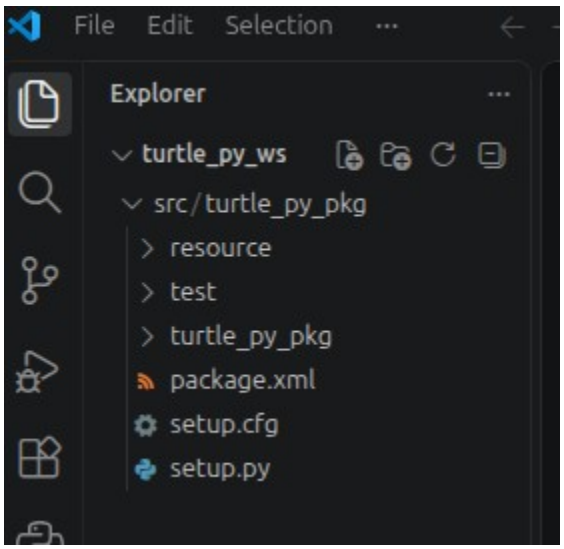
2. Running ros2 run turtlesim turtlesim_node



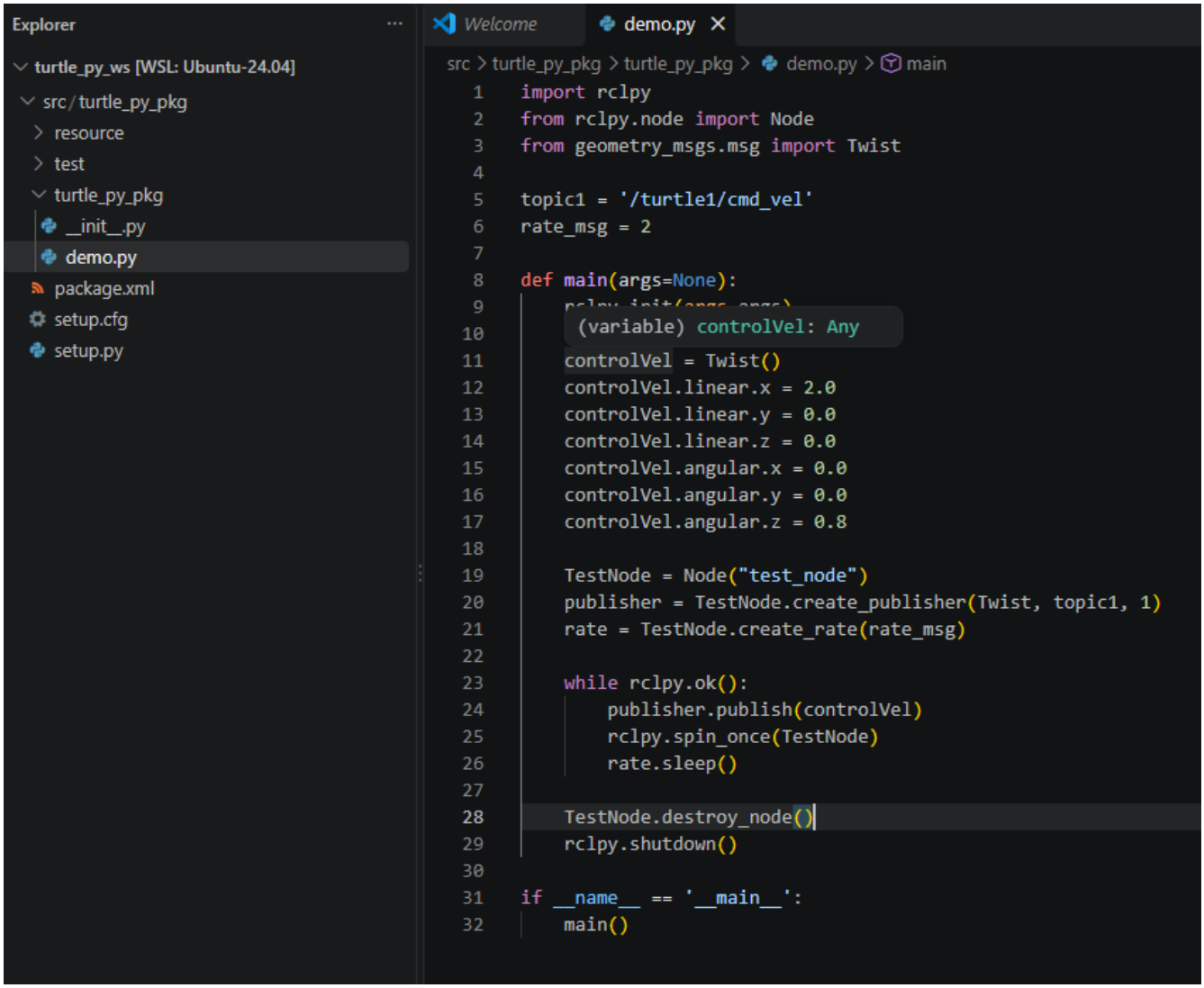
3. Creating the workspace and package:

```
mystic@Zallmos:~$ source /opt/ros/jazzy/setup.bash
mystic@Zallmos:~$ mkdir -p ~/turtle_py_ws/src
mystic@Zallmos:~$ cd ~/turtle_py_ws/src
mystic@Zallmos:~/turtle_py_ws/src$ ros2 pkg create --build-type ament_python turtle_py_pkg
going to create a new package
package name: turtle_py_pkg
destination directory: /home/mystic/turtle_py_ws/src
package format: 3
version: 0.0.0
description: TODO: Package description
maintainer: ['mystic <mystic@todo.todo>']
licenses: ['TODO: License declaration']
build type: ament_python
dependencies: []
creating folder ./turtle_py_pkg
creating ./turtle_py_pkg/package.xml
creating source folder
creating folder ./turtle_py_pkg/turtle_py_pkg
creating ./turtle_py_pkg/setup.py
creating ./turtle_py_pkg/setup.cfg
creating folder ./turtle_py_pkg/resource
creating ./turtle_py_pkg/resource/turtle_py_pkg
creating ./turtle_py_pkg/turtle_py_pkg/__init__.py
creating folder ./turtle_py_pkg/test
creating ./turtle_py_pkg/test/test_copyright.py
creating ./turtle_py_pkg/test/test_flake8.py
creating ./turtle_py_pkg/test/test_pep257.py
```

Step 4: Opening VS Code in turtle py folder



Step 5: Editing demo.py in VS Code.



Step 6: Editing package.xml in VS Code.

demo.py | package.xml

src > turtle_py_pkg > package.xml

```xml
<?xml version="1.0"?>
<?xml-model href="http://download.ros.org/schema/package_format
<package format="3">
  <name>turtle_py_pkg</name>
  <version>0.0.0</version>
  <description>TODO: Package description</description>
  <maintainer email="mystic@todo.todo">mystic</maintainer>
  <license>TODO: License declaration</license>

  <test_depend>ament_copyright</test_depend>
  <test_depend>ament_flake8</test_depend>
  <test_depend>ament_pep257</test_depend>
  <test_depend>python3-pytest</test_depend>

  <exec_depend>rclpy</exec_depend>
  <exec_depend>geometry_msgs</exec_depend>

  <export>
    <build_type>ament_python</build_type>
  </export>
</package>
```

Step 7: Editing setup.py in VS Code.

```
entry_points={
'console_scripts': [
    'publisher = turtle_py_pkg.demo:main',
    ],
},
```

Step 8: Build the Package from edited codes from steps 5-7

```
mystic@Zallmos:~/turtle_py_ws$ cd ~/turtle_py_ws
mystic@Zallmos:~/turtle_py_ws$ colcon build --packages-select turtle_py_pkg
Starting >>> turtle_py_pkg
Finished <<< turtle_py_pkg [2.48s]

Summary: 1 package finished [2.72s]
mystic@Zallmos:~/turtle_py_ws$ source install/setup.bash
```

Step 9: Running the Publisher

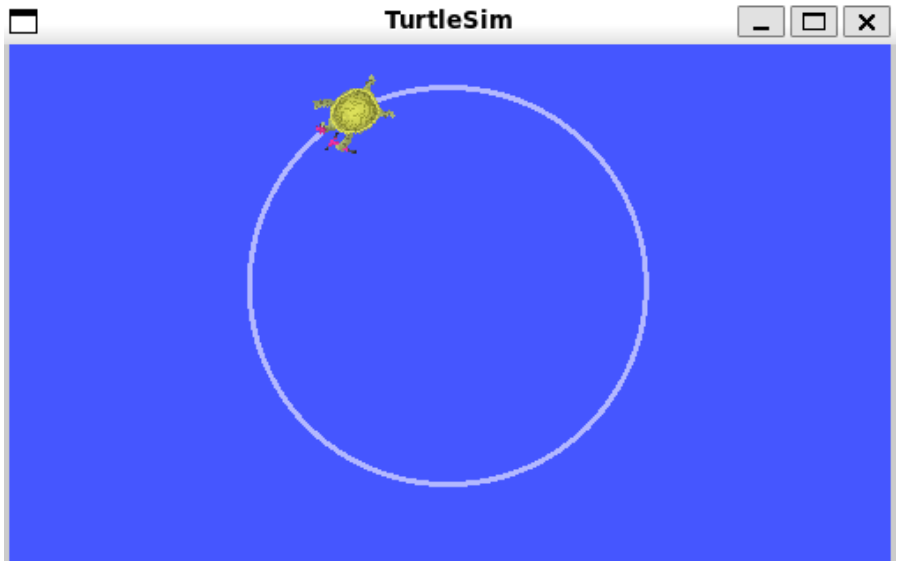
Terminal 1:

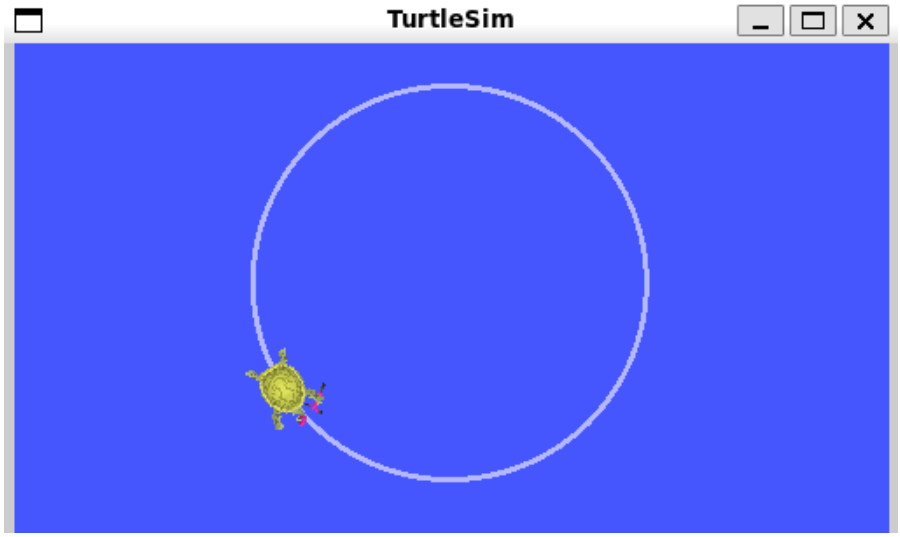
```
mystic@Zallmos:~$ source /opt/ros/jazzy/setup.bash
mystic@Zallmos:~$ ros2 run turtlesim turtlesim_node
[INFO] [1790618168.788418416] [turtlesim]: Starting turtlesim with node name /turtlesim
[INFO] [1790618168.803464909] [turtlesim]: Spawning turtle [turtle1] at x=[5.544445], y=[5.5444
45], theta=[0.000000]
```

Terminal 2:

```
mystic@Zallmos:~$ source ~/turtle_py_ws/install/setup.bash
mystic@Zallmos:~$ ros2 run turtle_py_pkg publisher
```

Step 10: Watching the turtle move





Additional Experiments:

Angular speed change: Editing demo.py in VS Code.

```python
import rclpy
from rclpy.node import Node
from geometry_msgs.msg import Twist

topic1 = '/turtle1/cmd_vel'
rate_msg = 2

def main(args=None):
    rclpy.init(args=args)

    controlVel = Twist()
    controlVel.linear.x = 2.0
    controlVel.linear.y = 0.0
    controlVel.linear.z = 0.0
    controlVel.angular.x = 0.0
    controlVel.angular.y = 0.0
    controlVel.angular.z = 0.4

    TestNode = Node("test_node")
    publisher = TestNode.create_publisher(Twist, topic1, 1)
    rate = TestNode.create_rate(rate_msg)

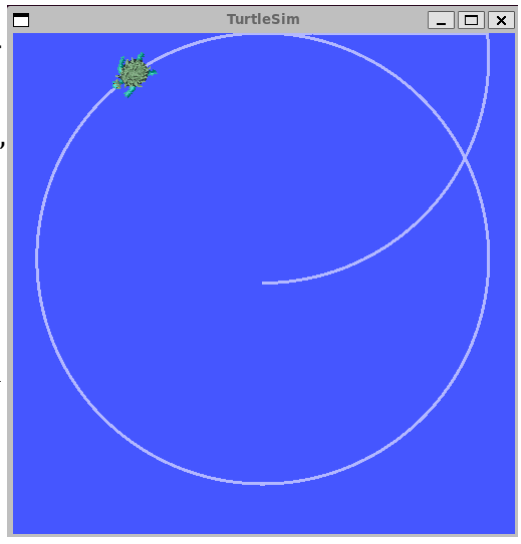
    while rclpy.ok():
        publisher.publish(controlVel)
        rclpy.spin_once(TestNode)
        rate.sleep()

    TestNode.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()
```

Rebuild and run again. Observe the turtle's motion. Describe the difference compared to the original 0.8 setting

Compared to the original controlVel.angular.z setting of 0.8, changing it to 0.4 affects the size of the circular path in the simulation. With the original 0.8 setting, the robot turns more sharply, resulting in a smaller circular path. When the value is reduced to 0.4, the robot turns more gradually, creating a larger circular path with a wider angle of travel. This difference is also noticeable when the robot approaches the simulation boundaries, which act like walls. With the 0.8 setting, the smaller circle keeps the robot closer to the center, while with 0.4, the wider circular path allows the robot to travel farther toward the boundaries.



Linear speed change: Editing demo.py in VS Code.

```python
import rclpy
from rclpy.node import Node
from geometry_msgs.msg import Twist

topic1 = '/turtle1/cmd_vel'
rate_msg = 2

def main(args=None):
    rclpy.init(args=args)

    controlVel = Twist()
    controlVel.linear.x = 4.0
    controlVel.linear.y = 0.0
    controlVel.linear.z = 0.0
    controlVel.angular.x = 0.0
    controlVel.angular.y = 0.0
    controlVel.angular.z = 0.8

    TestNode = Node("test_node")
    publisher = TestNode.create_publisher(Twist, topic1, 1)
    rate = TestNode.create_rate(rate_msg)

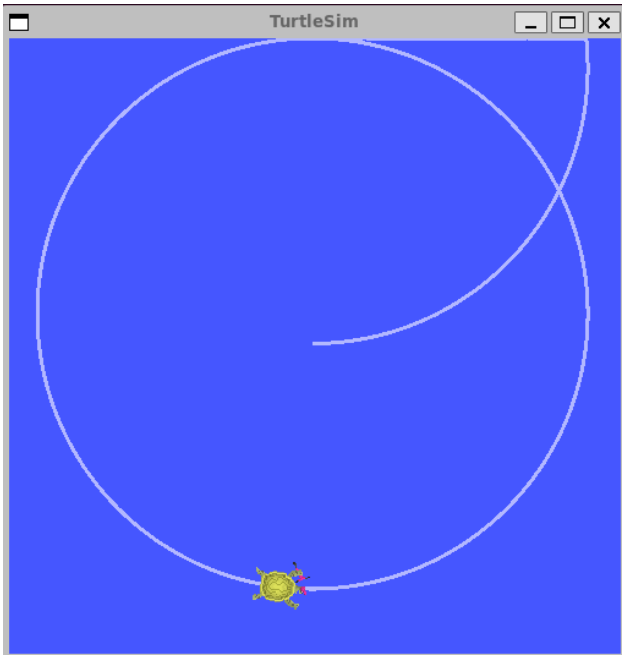
    while rclpy.ok():
        publisher.publish(controlVel)
        rclpy.spin_once(TestNode)
        rate.sleep()

    TestNode.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()
```

Rebuild and run again. Describe how the turtle's path changes compared to the default.

After rebuilding and running the simulation again, the turtle's path appears similar to the path observed when controlVel.angular.z was changed to 0.4. Compared to the default settings, the turtle follows a wider circular path and travels at a larger turning angle. The overall movement pattern remains circular, but the turtle covers a larger area of the simulation instead of making a tighter circle.



3. Answer the following questions:

1. In the ROS 2 publish/subscribe model, what role does the publisher node play and how does the subscriber receive data?

- The publisher node creates and sends messages containing data to a specific topic. For example, a ROS 2 node can publish a Twist message containing the turtle's linear and angular velocity.

- The subscriber listens to the topic and receives the messages published there. When a new message is published, the subscriber's callback function processes the data and uses it to perform an action.

2. Why must every new terminal running a ROS 2 node source the install/setup.bash file?

Each new terminal starts with its own shell environment. Running: source install/setup.bash loads the ROS 2 workspace's environment variables, packages, nodes, and executables so ROS 2 can find and run the nodes you built in that workspace.

3. What is the effect of changing the **linear** velocity vs. the **angular** velocity in the Twist message on the turtle's path?

- Linear velocity controls how fast the turtle moves forward. Increasing it makes the turtle travel faster and, for a given angular velocity, generally produces a larger-radius path.

- Angular velocity controls how quickly the turtle turns. Increasing it makes the turtle turn more sharply, producing a smaller circular path. Decreasing it produces a wider/larger circular path.